NovaCare

Hospital Management System

Subject: Software Design & Analysis

Group Members

Full Name	Roll Number
Yash Raj	24k-0737
Jiyanshu Raj	24k-0987

Faculty: Sir Jibran Rasheed Khan

Department: Department of Computer Science

University: National University of Computer & Emerging Sciences

Session: Spring 2025 – 2026

Year: 2nd Year, 4th Semester

1. Introduction

1.1 Project Idea

NovaCare is a comprehensive, web-based Hospital Management System developed using the Django framework (Python) with a relational SQLite3 database backend. The system unifies the core operational workflows of a modern hospital into a single, integrated digital platform: patient registration and appointment management, clinical consultation records, and a full-featured pharmacy with e-commerce and billing capabilities.

1.2 Problem Statement

Hospitals and clinics in developing regions continue to rely on paper-based records, fragmented spreadsheets, and manual billing processes. This leads to:

- Appointment scheduling conflicts and duplicate bookings
- Loss or inaccessibility of patient medical history and prescriptions
- Inventory shrinkage and stock-out situations in the pharmacy
- Delayed and inaccurate billing that erodes patient trust
- No consolidated view for administrators to manage doctors and services

NovaCare directly addresses each of these pain points through automated workflows, centralised records, and real-time stock management.

1.3 Domain / Application Area

- Healthcare Information Systems (HIS)
- Hospital & Clinic Administration
- Pharmacy Management & E-Commerce
- Electronic Medical Records (EMR)

1.4 Target Users (Beneficiaries)

Patients	Book appointments, browse the medicine catalogue, manage a shopping cart, and receive a unified bill combining consultation fees and pharmacy charges.
Doctors	View their appointment schedule, approve or cancel visits, and add full consultation notes including diagnosis, prescription, and test report remarks.
Pharmacists	Manage medicine inventory (CRUD), receive low-stock alerts, process over-the-counter POS bills, and track sales history.
Admin / Staff	Add, edit, and delete doctor profiles; manage hospital departments and available services through a staff-restricted admin panel.

2. Project Requirements

2.1 Technology Stack

Backend & Framework

- Python 3.14 — Core programming language
- Django 6.0.3 — Full-stack MVC/MVT web framework
- Django ORM — Database abstraction and query layer
- SQLite3 — Embedded relational database (development); upgradeable to PostgreSQL

Frontend & Templating

- Django Template Language (DTL) — Server-side HTML rendering
- HTML5 / CSS3 — Structure and styling
- JavaScript (Vanilla) — Client-side interactivity
- Responsive design via static/css/styles.css

Development Tools & Platforms

- VS Code / PyCharm — Integrated Development Environment
- Git & GitHub — Version control and collaboration
- pip — Python package manager
- Django Admin Panel — Built-in administration interface
- Postman — API and form endpoint testing (optional)
- draw.io / PlantUML — System design and UML diagramming

2.2 Hardware Requirements

- Minimum: 4 GB RAM, Dual-Core CPU, 10 GB disk space
- Recommended: 8 GB RAM, Quad-Core CPU, SSD
- Network: Localhost for development; LAN or cloud server for deployment

3. Functionalities and Features

NovaCare is designed around four independent yet deeply integrated modules. Below is a detailed breakdown of its functionalities and what distinguishes it from basic hospital applications.

3.1 Public-Facing Portal

- **Home Page:** Showcases hospital departments, hero banner, and quick navigation links
- **Doctor Directory:** Searchable and filterable listing of all registered doctors by name or department
- **Services Page:** Displays available hospital services (MRI, X-Ray, Blood Tests, etc.) with descriptions
- **Public Pharmacy:** Patients and visitors can search and browse the medicine catalogue without logging in
- **Branches & Contact:** Information pages for hospital locations and contact forms

3.2 Authentication & Role Management

- **Self-Registration:** Patients register via a form; a Patient profile is automatically created and linked
- **Role-Based Login:** A single login endpoint redirects users to their respective dashboard based on role flags
- **Django Admin:** Staff and superusers access the full admin panel for master data management
- **Secure Logout:** Session clearing with CSRF protection on all state-changing operations

3.3 Appointment Management

- **Smart Booking:** Patients select a doctor, date, and time; the system enforces slot uniqueness to prevent double-booking
- **Doctor Approval:** Doctors confirm or cancel appointments directly from their dashboard
- **Consultation Notes:** Doctors record full medical notes per appointment: diagnosis, prescription, and test report remarks
- **Status Lifecycle:** Appointments progress through Pending → Confirmed → Completed → (or Cancelled)
- **Medical History:** Doctors can view a complete chronological history of completed appointments for any patient

3.4 Pharmacy & Billing

- **Inventory CRUD:** Pharmacists add, edit, and delete medicines with formula, price, stock, expiry, and prescription flags
- **Low-Stock Alerts:** Dashboard automatically flags medicines with stock below 10 units
- **POS Billing (OTC):** Walk-in customers are billed via a point-of-sale interface; stock is automatically deducted
- **E-Commerce Cart:** Patients add medicines to a persistent cart with live stock validation and quantity management
- **Unified Checkout:** A single checkout combines pending consultation fees (doctor's fee) and cart medicines into one bill
- **Expenditure History:** Patients view all past UnifiedBills with itemised cost breakdowns
- **Automatic Reconciliation:** On checkout: stock is deducted, cart is cleared, and all pending appointments are marked as paid

What Makes NovaCare Unique: Unlike standalone appointment or pharmacy apps, NovaCare's Unified Billing Engine consolidates consultation and medicine charges in a single transaction, eliminating the need for patients to interact with separate billing counters. The role-based architecture ensures every user sees only the features relevant to their workflow.

4. System Modules

The system is divided into four primary Django applications (modules). Each app is self-contained with its own models, views, URL routes, and templates, yet communicates with other apps through well-defined foreign key relationships and shared views.

Module 1: Accounts Module (accounts app)

Manages all user identities and role-specific profile data. Acts as the identity backbone of the entire system.

- Custom User model extending Django AbstractUser with 4 role flags
- Doctor profile: department, fee, availability, experience, profile picture
- Patient profile: date of birth, blood group, phone, chronic conditions
- Pharmacist profile: experience and profile picture
- Department model for categorising doctors
- Role-based dashboard routing on login

Module 2: Appointments Module (appointments app)

Manages the full lifecycle of a patient-doctor appointment from booking to completed consultation record.

- Appointment model with unique-together constraint (patient, doctor, date, time)
- Status machine: Pending → Confirmed → Completed / Cancelled
- Doctor can approve, cancel, and add detailed consultation notes
- Medical record fields embedded in appointment: diagnosis, prescription, test reports
- Patient medical history view (filtered by completed appointments)
- is_paid flag integrated with Unified Billing checkout

Module 3: Core Module (core app)

Provides public-facing informational pages and admin tools for managing the hospital's doctor registry and services.

- Service model: MRI, X-Ray, Blood Tests, etc. with availability toggle
- Public doctor directory with search (name) and filter (department)
- Staff-restricted Add Doctor view: creates User + Doctor profile atomically
- Staff-restricted Edit Doctor and Delete Doctor (cascades to user account)
- Home, About, Services, Contact, and Branches informational pages

Module 4: Pharmacy Module (pharmacy app)

A dual-channel pharmacy system: a pharmacist-facing inventory and POS billing system, and a patient-facing e-commerce experience.

- Medicine CRUD with class-based views (ListView, CreateView, UpdateView, DeleteView)
- Stock update endpoint: add or remove units with validation
- POS Billing: create PharmacyBill + PharmacyBillItems, deduct stock automatically
- Patient e-commerce cart (PatientCart + PatientCartItem, one cart per patient)
- Unified Checkout: combines cart total + unpaid appointment fees into UnifiedBill
- Expenditure History: complete billing history for each patient

Module Interaction Summary

The four modules interact as described below:

Interaction	Description

Accounts ↔ Appointments	Appointment model holds FKs to Patient and Doctor (both from accounts). Views check doctor_profile / patient_profile attributes to enforce access control.
Accounts ↔ Pharmacy	PatientCart and UnifiedBill hold FKs to Patient. PharmacyBill holds FK to Pharmacist. Pharmacy views verify is_pharmacist and patient_profile flags.
Appointments ↔ Pharmacy	Unified Checkout queries Appointment to find unpaid consultations for the current patient and marks them is_paid=True on successful payment.
Core ↔ Accounts	Core views query Doctor and Department models from accounts to render the public doctor directory and staff management pages.

NovaCare — Hospital Management System

Department of Computer Science | National University of Computer & Emerging Sciences